Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 06 · Part B: Mathematics of Signals and Systems Level 2

Track: Z-Transform, Difference Equations, Pole-Zero Design, Discretisation, Discrete Control

Prerequisites: Problems 02, 05 · **Estimated effort:** 10–14 hours

Z-Transform and Discrete-Time Systems: The Mathematics of Everything That Runs on a Processor

Every filter and controller that runs on a flight computer is a difference equation. The z-transform turns it into a rational function whose poles and zeros you can place by hand: design a notch that deletes mains hum, a resonator that picks out one tone, map an analog design to digital in four different ways, and find the sample time at which a numerical integration or a controller blows up.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/06_z_transform_and_discrete_systems/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random

seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis sections, and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

A microcontroller does not integrate or differentiate; it adds and subtracts stored numbers once per clock tick. The discrete-time counterpart of the Laplace transform, the z-transform, is therefore the natural language of embedded signal processing. The unit circle plays the role of the imaginary axis, stability means poles *inside* the circle, and the frequency response is read off by walking around the circle.

You will also face the practical question that follows every analog design: "how do I run this on a computer?" There are several mappings from s to z, each with a different price in accuracy and stability. Choosing the sample rate and the mapping is a core embedded-systems skill, and this problem lets you see exactly what each choice does.

Theoretical Framework

1. Z-Transform, ROC and Transfer Function

$$X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n}, \qquad a^n u[n] \leftrightarrow \frac{1}{1 - a z^{-1}}, \ |z| > |a|, \qquad x[n-k] \leftrightarrow z^{-k} X(z). \tag{1}$$

A linear constant-coefficient difference equation $\sum_{k=0}^{N} a_k y[n-k] = \sum_{k=0}^{M} b_k x[n-k]$ has

$$H(z) = \frac{\sum_k b_k z^{-k}}{\sum_k a_k z^{-k}}, \qquad H(\mathrm{e}^{\mathrm{j}\omega}) = H(z)\big|_{z=\mathrm{e}^{\mathrm{j}\omega}}, \ \ \omega = 2\pi f / f_s. \tag{2}$$

A causal system is stable if and only if all poles satisfy $|p| < 1$. Poles near the unit circle at angle ω_0 create resonances; zeros on the unit circle at ω_0 create exact nulls.

2. Elementary Designs from Poles and Zeros

A *leaky integrator* $y[n] = a y[n-1] + x[n]$ has DC gain $1/(1-a)$. A *digital resonator* with poles at $r\mathrm{e}^{\pm \mathrm{j}\omega_0}$ has a 3 dB bandwidth of roughly $2(1-r)$ rad/sample for r close to 1. A *notch* that removes

frequency ω_0 places zeros at $\mathrm{e}^{\pm \mathrm{j}\omega_0}$ and poles at $r\mathrm{e}^{\pm \mathrm{j}\omega_0}$:

$$H_{\text{notch}}(z) = \frac{1 - 2\cos\omega_0\, z^{-1} + z^{-2}}{1 - 2r\cos\omega_0\, z^{-1} + r^2 z^{-2}}. \tag{3}$$

3. From Analog to Digital: Mappings

With sample time T and an analog $H(s)$:

- matched pole–zero / impulse invariance: poles map by $z = \mathrm{e}^{sT}$;
- forward Euler $s \to (z-1)/T$ and backward Euler $s \to (z-1)/(zT)$;
- bilinear (Tustin) $s \to \dfrac{2}{T}\dfrac{z-1}{z+1}$, which maps the whole left half-plane into the unit disc and warps frequency: $\omega_a = \dfrac{2}{T}\tan\dfrac{\omega_d T}{2}$, so the digital frequency ω_d of a feature is realised at analog frequency ω_a (*prewarp* to place a critical frequency exactly).

Forward Euler maps stable analog poles to $1 + sT$, which can leave the unit circle if T is too large: for the poles of a second-order system, $|1 + sT| < 1$ requires $T < 2\zeta/\omega_n$.

4. Delay in Digital Loops

A zero-order hold plus computation delay adds an effective delay of about T (hold alone $T/2$), i.e. a phase lag of $\omega T/2$ rad at frequency ω from the hold alone. A rule of thumb is $f_s \gtrsim 20$–30 times the closed-loop bandwidth.

Data and Simulation Specification

Sequences	$x[n] = a^n u[n]$ ($a = 0.8, -0.5, 1.1$), $x[n] = nu[n]$, $x[n] = \cos(\omega_0 n)u[n]$.
Difference equations	$y[n] = 0.9y[n-1] + x[n]$; $y[n] = 1.5y[n-1] - 0.7y[n-2] + x[n]$; 5-point moving average.
Mains notch	Sensor data at $f_s = 1000\,\text{Hz}$ with a $50\,\text{Hz}$ interference; `sp_common.synth.sensor_signal` plus a $50\,\text{Hz}$ sinusoid of amplitude 1 added to a $5\,\text{Hz}$ signal.
Analog prototype	$H(s) = \omega_n^2/(s^2 + 2\zeta\omega_n s + \omega_n^2)$ with $\omega_n = 2\pi \times 50\,\text{rad/s}$, $\zeta = 0.5$; sample times $T = 1\,\text{ms}, 5\,\text{ms}, 10\,\text{ms}, 15\,\text{ms}$.
Attitude loop	The $J = 10$, $\omega_n = 0.3$, $\zeta = 0.7$ satellite of problem 05, simulated with forward Euler at $T = 0.01, 0.1, 0.5, 2, 5\,\text{s}$ and discretised with Tustin.
Real data	Optional hardware: run your notch or resonator on a microcontroller (Arduino, ESP32) at a fixed sample rate with an ADC input, or process your own Phyphox accelerometer log sample by sample in a loop, as an embedded device would.

Your Task

1. **Transforms and inversion (Stage 2).** Derive the z-transforms of the sequences in the table, state each ROC and invert $X(z) = 1/((1 - 0.5z^{-1})(1 - 0.25z^{-1}))$ by partial fractions and by long division; check with `scipy.signal.residuez` and the impulse response of `lfilter`.
2. **Difference equations by hand.** Implement the difference equations as explicit loops (direct form I and transposed direct form II) and compare with `scipy.signal.lfilter` to machine precision. Plot pole-zero maps with the unit circle and the impulse responses; for $y[n] = ay[n-1] + x[n]$ sweep a across ± 1 and show the transition to instability at $|a| = 1$.

3. **Frequency responses.** Evaluate Eq. (2) on a frequency grid and compare with `scipy.signal.freqz`. Design (a) a digital resonator at 100 Hz, $f_s = 1000$ Hz with $r = 0.95$ and verify its bandwidth, and (b) the notch of Eq. (3) at 50 Hz and apply it to the sensor data; report the attenuation of the interference and the distortion of the 5 Hz signal for $r = 0.9, 0.95, 0.99$ together with the settling time of the transient.
4. **Mappings from s to z.** Discretise the analog prototype by matched poles, forward Euler, backward Euler and Tustin at each sample time. Plot step responses and magnitude responses against the analog original, and measure the frequency error of each method as a function of f/f_s. Show the warping of Tustin and prewarp it so that the cutoff is exact.
5. **Sample rate and stability.** Simulate the closed-loop attitude ODE with forward Euler at the listed T, find the largest stable T and compare with $2\zeta/\omega_n$. Repeat with Tustin and explain why it stays stable.
6. **Cascaded sections.** Realise a 4th-order Butterworth low-pass as one direct-form polynomial and as two cascaded second-order sections. Compare their responses in double precision and with coefficients rounded to 6 significant digits; explain the sensitivity.
7. **Sample-by-sample processing.** Write a stateful filter object with a `step(x)` method and show that processing a stream sample by sample, in blocks of 7 samples, and all at once gives identical output (use `lfilter`'s `zi`).

Self-check targets

The leaky integrator with $a = 0.9$ has DC gain 10; the notch has an exact zero at 50 Hz (magnitude $< 10^{-12}$ at that frequency in double precision). For forward Euler the $\omega_n = 0.3$, $\zeta = 0.7$ attitude loop must become unstable above $T = 2\zeta/\omega_n = 4.67$ s; the Tustin version is stable for every T. The Tustin map of an analog frequency ω_a is $\omega_d = 2\arctan(\omega_a T/2)$.

Aerospace Application Link

A flight computer runs the attitude loop of problem 05 at a fixed rate. Using your Task 5 results and the delay rule of thumb (hold plus compute latency $\approx T$), compute the loop's phase margin loss for $f_s = 10$, 50, 200 Hz given a gain-crossover frequency of 0.5 rad/s, and recommend the smallest sample rate that costs less than 5° of phase. State what additional latency from a star tracker running at 5 Hz does to this budget.

Deliverables

D1 **Transform notebook:** z-transforms, ROC statements, inversions with checks.

D2 **Difference-equation implementations:** loops versus `lfilter`, pole-zero maps, stability sweep.

D3 **Resonator and notch designs:** frequency responses, attenuation and settling-time table for the three r values.

D4 **Mapping comparison:** step and magnitude responses for the four methods and four sample times, with the prewarped Tustin design.

D5 **Sample-rate study:** the attitude-loop stability limits and the cascade-versus-direct sensitivity comparison, plus the streaming-versus-block equivalence test.

D6 **Memo (250 words):** "What sample rate should the attitude loop run at?" Cite at least two external sources.

Bonus (a) Derive the exact relation between pole radius and the 3-dB bandwidth of the resonator. (b) Implement the filters in fixed-point integer arithmetic. (c) Run the notch on a real microcontroller reading a signal generator and measure its attenuation with an oscilloscope.

Learning Resources

Z-Transform and Discrete Systems

- Oppenheim, A.V., Schafer, R.W. *Discrete-Time Signal Processing*, Ch. 2–3 (discrete-time systems, z-transform) and Ch. 5 (transform analysis of LTI systems).
- Proakis, J.G., Manolakis, D.G. *Digital Signal Processing*, Ch. 3–5.
- Lyons, R.G. *Understanding Digital Signal Processing*, Ch. 6 (IIR filters) for the pole-zero intuition.

Discretisation and Control

- Franklin, G.F., Powell, J.D., Workman, M.L. *Digital Control of Dynamic Systems* (emulation of continuous designs, delay from sampling).
- Åström, K.J., Wittenmark, B. *Computer-Controlled Systems* (sampling and discretisation methods).

Tools

- SciPy `scipy.signal` (`lfilter`, `freqz`, `residuez`, `cont2discrete`): https://docs.scipy.org/doc/scipy/reference/signal.html

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.